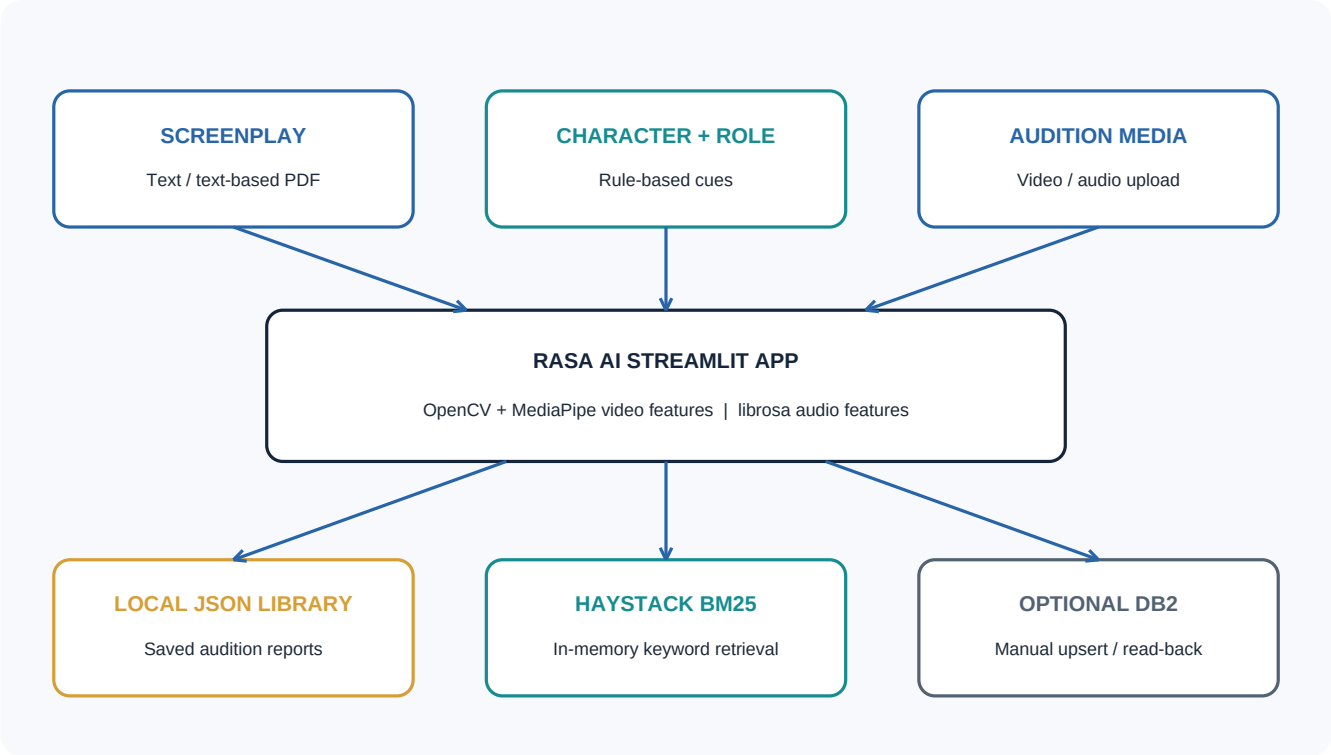
RASA AI

IMPLEMENTATION ARCHITECTURE DOCUMENT

Audition intelligence and casting-support application



Document type: Working implementation architecture
Scope: Features and integrations visible in the reviewed RASA AI source code and README
Source snapshot: app.py, smart_audition_search.py, db2_store.py and README.md (29 October 2026)
Version: 2.0 | **Date:** 30 October 2026

Scope discipline: This document describes implemented application behavior only. Optional, experimental and unverified capabilities are labelled as such. It does not describe Rasa Open Source/Pro platform internals because the reviewed project is a Streamlit application, not a Rasa framework deployment.

1. Purpose and implementation scope

RASA AI is a Streamlit-based audition review workspace. It organizes screenplay context, helps reviewers define role requirements, extracts observable visual and vocal evidence from audition media, stores audition reports locally, and retrieves saved records using Haystack BM25 keyword search.

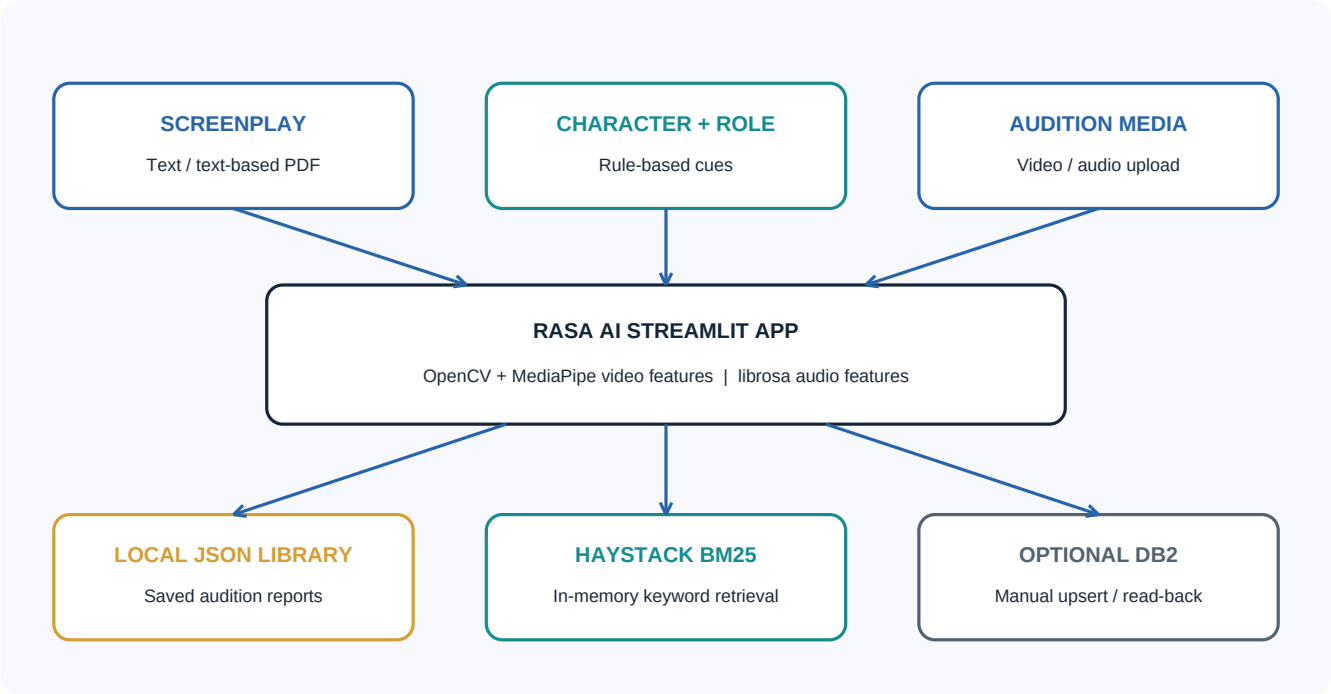
The application is a human decision-support tool. Its measurements and retrieval scores are descriptive evidence for review; they do not make casting decisions or establish acting quality, personality, internal emotion or role suitability.

Implemented workflow

Stage	What the application does	Implementation status
Screenplay	Accepts screenplay/story text or a text-based PDF; extracts selectable PDF text and applies deterministic rules to detect character cue candidates and scene headings.	Implemented; heuristic
Character cues	Finds nearby keyword evidence around character names/cue lines and suggests audition-test prompts.	Implemented; heuristic
Role requirements	Lets a reviewer define role targets and compares selected targets with available video measurements.	Implemented; rule-based
Video analysis	Processes sampled video frames and calculates pose/landmark and facial-geometry measurements when landmarks are available.	Implemented; environment-dependent
Vocal analysis	Runs optional audio-only acoustic analysis, including pitch and RMS energy summaries and timestamped evidence.	Optional; dependency-dependent
Evidence timeline	Aligns available video and audio observations by timestamps using a configurable offset/tolerance; exports CSV/JSON.	Implemented when both reports exist
Audition library	Saves, reloads, reviews, filters, compares and exports audition reports; supports reviewer notes, status and shortlist markers.	Implemented locally
Haystack retrieval	Searches saved audition report fields and sampled observations with BM25 using an in-memory document store.	Implemented; keyword retrieval
Db2 connector	Checks connectivity, initializes RASA_AUDITIONS, manually upserts local records and loads recent rows.	Optional; live connection not verified

2. Application architecture

The main application is app.py. Streamlit presents the workflow as eight tabs. The application uses local JSON persistence for its audition library; Haystack search builds an in-memory index from the saved records when a search is run. Db2 is a separate optional persistence connector and is not the current source of truth.



Data flow in brief

1. A reviewer enters screenplay text or uploads a text-based PDF.
2. Rule-based parsing identifies candidate character cues and scene-heading counts.
3. The reviewer defines role requirements and uploads an audition video and/or audio.
4. Video and audio analysis produce separate observable measurements and timestamped evidence.
5. The reviewer saves audition reports to the local JSON library and can add review metadata.
6. Haystack BM25 searches the saved records; optionally, the reviewer manually syncs records to Db2 and reads them back.

> There is no implemented general-purpose Rasa orchestrator, autonomous agent framework, LLM response-generation layer, or semantic vector database in the reviewed application source.

3. Screenplay and role-requirement processing

Screenplay input

The Screenplay workflow accepts typed story/screenplay text and PDF uploads. PDF extraction uses pypdf and extracts selectable text. The code counts scene headings beginning with INT./EXT.-style prefixes. It does not include OCR for scanned/image-only PDFs.

Character cue extraction

parse_screenplay() looks for short, mostly uppercase lines that resemble screenplay character cues. Users can also supply character names manually. It excludes common scene headings and transition terms, counts name mentions, and creates a profile for each candidate. The role description is not inferred automatically.

extract_character_specific_cues() examines text near a character name or standalone cue line and searches a fixed keyword list for movement, gestures, stillness, physical tension, visible expression changes and dialogue-heavy cues. Matches include evidence text and suggested audition-test prompts. These are proximity-based heuristics and require manual verification.

Role requirements

The reviewer selects role targets. The implementation compares a subset of targets against available video report fields, including mean normalized landmark movement, fraction of samples with a wrist above shoulder level and mean shoulder-tilt index. Thresholds in the code are exploratory rule thresholds, not validated performance standards.

Role target	Video field used	Code threshold
More visible body movement / controlled stillness	Mean normalized landmark movement index	0.025
Frequent hands above shoulder level	Fraction of sampled frames with wrist above shoulder	0.25
More level shoulder alignment	Mean shoulder tilt index	0.04

These comparisons describe measured features only. They do not calculate a talent score or automatically rank candidates.

4. Video, vocal and combined evidence

Video analysis

The Video Analysis workflow accepts an audition video and actor/audition label. The code uses OpenCV for frame handling and MediaPipe landmarks for observable pose/body and facial-geometry measurements when detection succeeds. It records sampled observations with timestamps and evidence-quality information. Optional FER-based facial-expression labels can be enabled when the package/model is available; these labels are experimental and may be inaccurate.

The code includes measurements such as pose/face detection rates, normalized landmark movement, shoulder tilt, torso offset, wrist-above-shoulder fraction, mouth-opening ratio, eye-openness proxy and brow/eye-spacing proxy. These are geometric or tracking features, not validated emotion or psychological measurements.

Vocal analysis

The Vocal Analysis workflow is optional and depends on the installed audio-analysis packages. It analyzes audio separately from video and produces acoustic summaries such as pitch (Hz), RMS signal energy and timestamped samples. It supports time-segment summaries and CSV/JSON downloads. The reviewed implementation does not perform speech transcription and does not establish the speaker's internal emotional state.

Combined evidence timeline

When both video and vocal reports are available, the Evidence Timeline combines their timestamped observations using a reviewer-specified audio offset and matching tolerance. It displays a combined table and provides CSV/JSON exports. The code warns that the tool aligns measurements in time; it does not verify lip-sync or infer emotional meaning.

> Media quality, lighting, framing, camera angle, occlusion, audio quality and tracking success can affect the measurements. Review the original recording alongside the derived evidence.

5. Local audition library and review workflow

The local library is stored in rasaai_audition_library.json beside the application. app.py loads it into Streamlit session state and writes updated records back to the JSON file. The code uses a temporary JSON file followed by replacement when saving, and creates library snapshots/backups as part of its local data-management helpers.

Saved audition records can be reviewed and compared. The application includes reviewer-facing fields such as actor label, selected character, reviewer decision/status, shortlist marker and reviewer notes. These are human-managed review inputs; the shortlist marker is not an automated casting recommendation.

The app also includes local health checks, activity logging, library inspection, backup/restore helpers and export functions. These operate on local application data; they should not be confused with a central multi-user database or a permanent audit service.

Persistence boundaries

Data	Where it lives	Persistence behavior
Audition reports	Local JSON file: rasaai_audition_library.json	Persists across local app restarts when saved successfully.
Current workflow/search state	Streamlit session state	Session-scoped; not a separate durable database.
Haystack index	In-memory document store created during search	Rebuilt from the supplied saved audition items for each search call.
Db2 records	Optional Db2 LUW table RASA_AUDITIONS	Manual upsert and read-back; not automatically the app's source of truth.

6. Haystack retrieval implementation

The helper smart_audition_search.py uses Haystack 2.x components: Document, InMemoryDocumentStore and InMemoryBM25Retriever. For each search, it converts saved audition fields into searchable text, creates one Haystack Document per record, writes those documents to a new in-memory store and runs BM25 retrieval.

Searchable content includes actor label, video filename, selected character, review status, shortlist flag, reviewer notes, duration, detection rates, selected movement/facial-geometry measurements, evidence-quality notes, analysis method and up to 100 timestamped observations per report.

The helper returns the original library index, BM25 score and indexed text. The UI uses the index to display the corresponding saved audition. A higher BM25 score indicates stronger keyword relevance to the query within this search—not acting quality, role fit or casting probability.

Important implementation detail

The document store is in memory and is rebuilt when search_auditions() runs. The project does not implement a persistent Haystack index, vector embeddings, semantic similarity search or an LLM-based retrieval answer generator in this helper.

Search history shown by the UI is session-only. An empty local audition library produces no audition matches.

7. Optional Db2 integration

The Db2 integration is implemented in db2_store.py and is exposed through the Db2 Database tab in app.py. It is optional: the local JSON audition library remains the application's source of truth. The connector reads connection settings from environment variables and uses the ibm_db Python driver.

Configuration

Required environment variables: DB2_HOST, DB2_DATABASE, DB2_USER and DB2_PASSWORD. DB2_PORT defaults to 50000. DB2_SSL can enable SSL when the Db2 service requires it. Credentials should not be hard-coded or committed to source control.

Implemented operations

Operation	Implementation
Connection check	Opens a Db2 connection and runs VALUES CURRENT SERVER; returns the server name when successful.
Schema initialization	Checks SYSCAT.TABLES for RASA_AUDITIONS in the current schema; creates the table if absent.
Manual sync	MERGE upserts each local audition record by AUDITION_ID. Stores actor, character, review status, shortlist flag and full JSON payload in a CLOB.
Read-back	Fetches recent rows ordered by UPDATED_AT and decodes the JSON payload.

The sync does not delete Db2 rows and does not automatically redirect the rest of the application to Db2. The README explicitly states that a live cloud connection has not yet been verified and that this initial connector uses Db2 LUW catalog SQL; compatibility with other Db2 editions may require changes.

8. Technology and project files

Technology / file	Actual use in this project
Python	Application and helper implementation language.
Streamlit	Local web UI and workflow tabs.
pypdf	Extracts selectable text from PDF screenplay uploads.
OpenCV	Video frame handling.
MediaPipe	Pose/landmark detection used for visual measurements.
NumPy / pandas	Numerical processing and tabular evidence handling.
librosa	Optional vocal/acoustic analysis.
FER	Optional experimental facial-expression labels.
Haystack 2.x	In-memory BM25 keyword retrieval over saved audition reports.
ibm_db	Optional Db2 LUW connectivity.
app.py	Main Streamlit application and workflow tabs.
smart_audition_search.py	Haystack BM25 retrieval helper.
db2_store.py	Db2 connection, table initialization, sync and fetch helpers.
requirements.txt	Python dependency declarations.

The source project also references local JSON data and generated exports. Keep the application files and required helper modules together in the project directory, as described in the README.

9. Current limitations and verification status

Confirmed implementation limitations

• Screenplay parsing is deterministic and rule-based; character detection and cue matching are heuristic.
• Scanned/image-only screenplay PDFs are not OCR-processed.
• Video measurements depend on visible and successfully tracked landmarks.
• Optional FER expression labels are experimental and can be inaccurate.
• Vocal analysis describes acoustic features; it is not speech transcription or reliable emotion recognition.
• Haystack BM25 is keyword retrieval, not semantic/vector retrieval.
• Haystack's document store is in memory and rebuilt for a search.
• Search history is session-only.
• Db2 is optional; local JSON remains the source of truth and sync is manual.
• A live Db2 cloud connection has not been verified in the project README.

Not represented as implemented

The reviewed source does not support describing this application as a Rasa Open Source/Pro deployment, LLM-powered conversational agent, autonomous multi-agent system, persistent vector database, Kubernetes deployment, or production-connected enterprise integration platform. Those elements are intentionally excluded from this document.

Validation still required

Before treating this as a production architecture, test the complete workflow in the target environment: dependency installation, PDF/text parsing, video/audio processing, local data backup/restore, Haystack retrieval with representative records, and Db2 connectivity/schema/sync/read-back against the actual Db2 instance.

Source basis

This document was prepared from the reviewed RASA AI README.md, app.py snapshot dated 29 October 2026, smart_audition_search.py and db2_store.py. It describes the source snapshot and should be updated when implementation changes.

End of document